

Receptes Könyv specifikáció

Tran Duy Dat

GS9HDY

1. A program célja

A program célja egy egyszerű, konzolos felületű receptkezelő alkalmazás létrehozása. A szoftver lehetővé teszi a felhasználó számára, hogy saját recepteket rögzítsen, azokat kategóriába rendezze, elmentse egy külső fájlba majd módosítani is tudja azt vagy keressen benne.

2. Funkcionális követelmények

1. Receptkezelés: Új recept felvétele névvel, hozzávalókkal, leírással, elkészítési idővel.
2. Adattárolása: Új receptek mentése egy fájlba (receptek.txt)
3. Adatbetöltés: A fájlban a beolvasása, hogy a meglévő recepteket az alkalmazás megtudja jeleníteni.
4. Keresés: Receptek közötti keresés.
5. Listázás: Az összes recept kilistázása.
6. Módosítás: A kiválasztott recept módosítása.
7. Törlés: Egy recept kitörlése a könyvből.

3. Nem-funkcionális követelmény

1. Rossz inputok esetén kezeli a program és jelzi azt a felhasználónak
2. Hiba esetén jelzi azt a felhasználónak.
3. A program egyetlen futtatható állomány (Windowson .exe build után)

4. Osztályok:

1. Recipe osztály
2. RecipeBook osztály
3. Ingredient osztály
4. FileManager osztály
5. MenuSystem osztály

5. Könyvtár struktúra

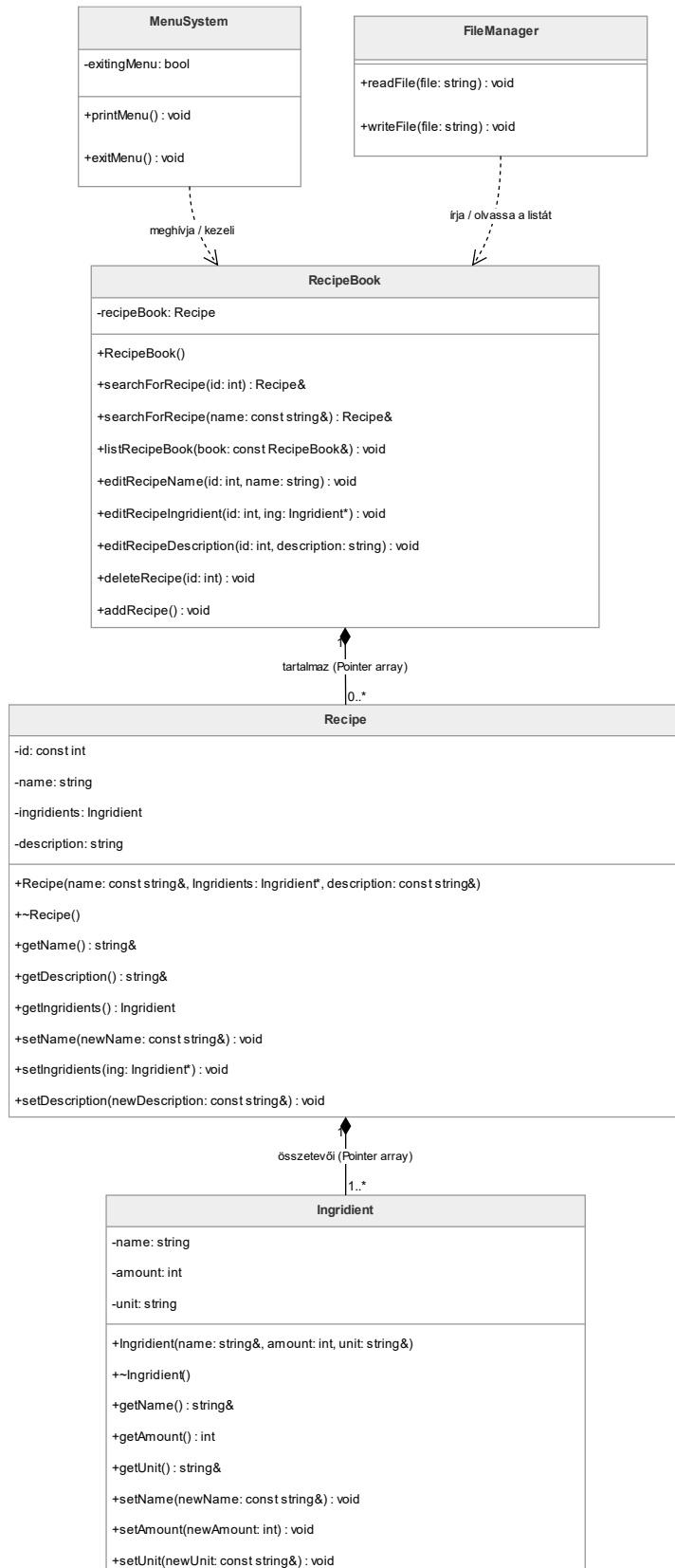
1. Minden classnak külön mappa amiben benne van a .h illetve .cpp fájl
2. Rootban található meg a recipes.txt ami a recepteket tartalmazza
3. TEST file is lesz a program teszteléséhez

6. Input file REGEX:

1. A recepteket ilyen formában fogom eltárolni és kiolvasni a fájlból:

NÉV;HOZZAVALÓ1:MENNYISEG:MERTEKEGYSEG;HOZZAVALÓ2:MENNYISEG:MERTEKEGYSEG;LEÍRÁS

7. Osztály diagram:



8. Osztályok leírásai és rövid kódjai

1. `RecipeBook` osztály

i. A recepteket tárolja és kezeli

1. `addRecipe()` → recept hozzáadása
2. `deleteRecipe(int id)` → recept törlése id alapján
3. `searchForRecipe(int id)` és `searchForRecipe(const string& name)` → recept keresése id vagy név alapján
4. `editRecipeName(int id, string name)` (és a többi edit függvény) → recept elemek módosítása

2. `FileManager` osztály

i. Ez az osztály a fájlt kezeli

1. `readFile(const string file)` → fájl beolvasása
2. `writeFile(const string file)` → fájl írása

3. `MenuSystem` osztály

i. A kiíratásért felelős osztály ez a UI része

1. `printMenu()` → menu kiíratása
2. `exitMenu()` → kilépés a menüből

4. `Recipe` osztály

i. A receptek adatait tároló osztály

1. Getterek és setterek vannak benne → adattagok lekérése és beállítása

5. `Ingredient` osztály

i. A hozzávalók adatait tároló osztály

1. Getterek és setterek vannak benne → adattagok lekérése és beállítása

9. Függvények phszeudó kódjai

Függvény `addRecipe()`:

Először bekérem a konzolról az új recept nevét és leírását.

Megkérdezem a felhasználótól, hogy hány hozzávaló szükséges a recepthez.

Mivel előre nem tudom a pontos számot, lefoglallok a memóriában egy új, dinamikus Hozzávaló (Ingredient) tömböt a megadott mérettel.

Elindítok egy ciklust, ami annyiszor fut le, ahány hozzávalót megadtak:

Minden lépésnél bekérem az aktuális hozzávaló nevét, mennyiségét és mértékegységét.

Az adatokat elmentem a dinamikus tömböm aktuális elemébe.

Amikor megvan minden adat, létrehozok egy új Recept objektumot, és átadom neki a nevet, a leírást, és a hozzávalókat tartalmazó tömb mutatóját.

Végül ezt az új Recept objektumot elhelyezem a 'recipeBook' tömböm következő szabad helyére (ha kell, előtte megnövelem a tömb méretét).

Függvény `deleteRecipe(id)`:

Végigmegyek a 'recipeBook' tömbömon, és keresem azt a receptet, aminek az azonosítója megegyezik a kapott 'id'-vel.

Ha megtaláltam a receptet (például az 'i'-edik indexen):

Először felszabadítom a memóriát, amit ez a recept a hozzávalóinak (a belső dinamikus tömbjének) lefoglalt, hogy ne legyen memóriaszivárgás.

Ezután elindítok egy ciklust az 'i'-edik indextől a tömböm végéig.

Minden elemet eggyel balra csúsztatok (tehát a `receptek[j] = receptek[j+1]`), így eltüntetem a "lyukat", amit a törölt recept hagyott.

Végül eggyel csökkentem a nyilvántartott receptek számát.

Ha a ciklus végéig nem találtam meg a receptet:

Egyszerűen kiírom a konzolra: "Hiba: A megadott azonosítójú recept nem található."

Függvény `searchForRecipe(keresett_ertek)`:

Elindítok egy ciklust, ami végigmegy a 'recipeBook' tömböm összes eddig felvett elemén.

Minden lépésnél ellenőrzöm:

Vajon az aktuális recept azonosítója (vagy neve) megegyezik a paraméterben kapott 'keresett_ertek'-kel?

Ha igen, akkor azonnal befejezem a keresést, és visszatérek a megtalált recepttel.

Ha a ciklus lefutott, és idáig jutottam, az azt jelenti, hogy nem volt találat.

Kiírom a konzolra: "Keresési hiba: Nincs ilyen recept."

Visszatérek valamilyen hibaértékkel (például egy üres, érvénytelen Recept objektummal, mivel referenciát kell visszaadnom az eredeti kódod szerint, ami amúgy veszélyes).

Függvény `editRecipeName(id, uj_nev)`:

Először megkeresem a receptet a tömbömben az 'id' alapján (akár egy belső ciklussal, vagy a `searchForRecipe` felhasználásával).

Ha megtaláltam a receptet:

Meghívom a megtalált recept `setName()` metódusát, és átadom neki a paraméterben kapott 'uj_nev'-et.

Ha nem találtam meg:

Kiírom a konzolra: "Szerkesztési hiba: A recept nem található."

Függvény `readFile(fajlnev)`:

Megpróbálom megnyitni a paraméterben kapott nevű fájlt olvasásra.

Ha a megnyitás sikertelen (pl. nem létezik a fájl):

Kiírom a konzolra: "Fájlolvasási hiba: A megadott fájl nem nyitható meg."

Befejezem a függvény futását (return).

Ha a fájl sikeresen megnyílt:

Elkezdek egy ciklust, ami addig fut, amíg el nem érem a fájl végét.

Minden iterációban beolvasom a következő recept adatait a fájlból.

Ahogy felépítem az adatokat a memóriában, létrehozom a `Recept` objektumot, és hozzáadom a `RecipeBook`-hoz.

Amikor a fájl végére értem, bezárom a fájlt.

Függvény `writeFile(fajlnev)`:

Megpróbálom megnyitni a paraméterben kapott nevű fájlt írásra.

Ha a megnyitás sikertelen:

Kiírom a konzolra: "Fájlírási hiba: A fájl nem menthető."

Befejezem a függvény futását (return).

Ha a fájl sikeresen megnyílt:

Lekérem a `RecipeBook`-tól az összes receptet, és elindítok rajtuk egy ciklust.

Minden receptnél kiírom a fájlba a recept nevét, leírását és a hozzávalókat megfelelően formázva.

Amikor minden receptet kiírtam, lezárom a fájlt.

Függvény `printMenu()`:

Elindítok egy ciklust, ami addig fut, amíg az `'exitingMenu'` belső változóm HAMIS.

A ciklus minden lépésében:

Kiírom a képernyőre az elérhető opciókat (pl. "1. Új recept, 2. Listázás, 3. Keresés, 4. Kilépés").

Várok, amíg a felhasználó beír egy választást.

Megvizsgálom a beírt választást:

Ha az érték 1: Meghívom a `RecipeBook addRecipe()` metódusát.

Ha az érték 2: Meghívom a `RecipeBook listRecipeBook()` metódusát.

Ha az érték 3: Bekérem a keresendő adatot, és meghívom a `searchForRecipe()` metódust.

Ha az érték 4: Meghívom az `exitMenu()` metódusomat.

Egyéb esetben: Kiírok egy figyelmeztetést a konzolra: "Érvénytelen választás".

Getter Függvények pl.:

Függvény `getName()`:

Egyszerűen csak visszaadom annak a belső, privát `'name'` változónak az értékét, amit ez az objektum tárol.

Setter függvények pl.:

Függvény `setName(newName)`:

Kicserélem az objektumom belső, privát `'name'` változójának az értékét arra a szövegre, amit a `'newName'` paraméterben megkaptam.
